BITS PILANI, DUBAI CAMPUS
DUBAI INTERNATIONAL ACADEMIC CITY, DUBAI
SECOND SEMESTER 2025 - 2026

Data Structures & Algorithms (CS F211)

```c
/*
Name: Kirtika
Roll No: 2023A7PS0219U
Course: CS F211 - Data Structures & Algorithms
Lab Exercise 6: Merge Sort on Employee Records
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/time.h>

#define MAX_RECORDS 200
#define NAME_LEN 64

typedef struct {
    char name[NAME_LEN];
    int id;
    float salary;
} Employee;

static long long current_time_us(void) {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return (long long)tv.tv_sec * 1000000LL + (long long)tv.tv_usec;
}

static void print_records(const Employee arr[], int n) {
    printf("%-5s %-20s %-10s %-10s\n", "No.", "Name", "Emp_ID", "Salary");
    printf("--------------------------------------------------\n");
    for (int i = 0; i < n; i++) {
        printf("%-5d %-20s %-10d %-10.2f\n", i + 1, arr[i].name, arr[i].id,
arr[i].salary);
    }
}

static void print_name_stage(const Employee arr[], int n, int left, int right) {
    printf("Intermediate names after merge [%d..%d]: ", left, right);
    for (int i = 0; i < n; i++) {
        printf("%s", arr[i].name);
        if (i != n - 1) {
```

```
            printf(" ");
        }
    }
    printf("\n");
}

static void merge(Employee arr[], int left, int mid, int right, int total_size) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    Employee *left_arr = (Employee *)malloc((size_t)n1 * sizeof(Employee));
    Employee *right_arr = (Employee *)malloc((size_t)n2 * sizeof(Employee));

    if (left_arr == NULL || right_arr == NULL) {
        free(left_arr);
        free(right_arr);
        fprintf(stderr, "Memory allocation failed during merge.\n");
        exit(1);
    }

    for (int i = 0; i < n1; i++) {
        left_arr[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++) {
        right_arr[j] = arr[mid + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = left;

    while (i < n1 && j < n2) {
        if (strcmp(left_arr[i].name, right_arr[j].name) <= 0) {
            arr[k++] = left_arr[i++];
        } else {
            arr[k++] = right_arr[j++];
        }
    }

    while (i < n1) {
        arr[k++] = left_arr[i++];
    }

    while (j < n2) {
        arr[k++] = right_arr[j++];
    }

    print_name_stage(arr, total_size, left, right);

    free(left_arr);
```

```c
    free(right_arr);
}

static void merge_sort(Employee arr[], int left, int right, int total_size) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        merge_sort(arr, left, mid, total_size);
        merge_sort(arr, mid + 1, right, total_size);
        merge(arr, left, mid, right, total_size);
    }
}

int main(void) {
    Employee employees[MAX_RECORDS];
    int n;

    printf("Enter number of employee records (at least 10, max %d): ",
MAX_RECORDS);
    if (scanf("%d", &n) != 1 || n < 10 || n > MAX_RECORDS) {
        printf("Invalid number of records.\n");
        return 1;
    }

    printf("Enter each record as: name id salary\n");
    for (int i = 0; i < n; i++) {
        printf("Record %d: ", i + 1);
        if (scanf("%63s %d %f", employees[i].name, &employees[i].id,
&employees[i].salary) != 3) {
            printf("Invalid input format.\n");
            return 1;
        }
    }

    printf("\nUnsorted employee records:\n");
    print_records(employees, n);

    long long start = current_time_us();
    merge_sort(employees, 0, n - 1, n);
    long long elapsed = current_time_us() - start;

    printf("\nSorted employee records (ascending by name):\n");
    print_records(employees, n);

    printf("\nMerge sort execution time: %lld micro-secs\n", elapsed);
    return 0;
}
```

OUTPUT

```
manavaryasingh@MacBook-Pro-5 kritika-dsa %  cd "/Users/manavaryasingh/kritika-dsa" && gcc "week 6/ex6_merge_sort_employees.c" -o "week 6/ex6" && "week 6/ex6"
10
Ravi 3001 45000.00
Anita 3002 52000.50
Bimal 3003 47000.00
Deepa 3004 47000.00
Charan 3005 43000.75
Esha 3006 60000.00
Farhan 3007 39000.00
Gita 3008 52000.50
Hema 3009 47000.00
Ishan 3010 48000.25
EOF
Enter number of employee records (at least 10, max 200): Enter each record as: name id salary
Record 1: Record 2: Record 3: Record 4: Record 5: Record 6: Record 7: Record 8: Record 9: Record 10:
Unsorted employee records:
No.   Name                 Emp_ID     Salary
-----------------------------------------------
1     Ravi                 3001       45000.00
2     Anita                3002       52000.50
3     Bimal                3003       47000.00
4     Deepa                3004       47000.00
5     Charan               3005       43000.75
6     Esha                 3006       60000.00
7     Farhan               3007       39000.00
8     Gita                 3008       52000.50
9     Hema                 3009       47000.00
10    Ishan                3010       48000.25
Intermediate names after merge [0..1]: Anita Ravi Bimal Deepa Charan Esha Farhan Gita Hema Ishan
Intermediate names after merge [0..2]: Anita Bimal Ravi Deepa Charan Esha Farhan Gita Hema Ishan
Intermediate names after merge [3..4]: Anita Bimal Ravi Charan Deepa Esha Farhan Gita Hema Ishan
Intermediate names after merge [0..4]: Anita Bimal Charan Deepa Ravi Esha Farhan Gita Hema Ishan
Intermediate names after merge [5..6]: Anita Bimal Charan Deepa Ravi Esha Farhan Gita Hema Ishan
Intermediate names after merge [5..7]: Anita Bimal Charan Deepa Ravi Esha Farhan Gita Hema Ishan
Intermediate names after merge [8..9]: Anita Bimal Charan Deepa Ravi Esha Farhan Gita Hema Ishan
Intermediate names after merge [5..9]: Anita Bimal Charan Deepa Ravi Esha Farhan Gita Hema Ishan
Intermediate names after merge [0..9]: Anita Bimal Charan Deepa Esha Farhan Gita Hema Ishan Ravi

Sorted employee records (ascending by name):
No.   Name                 Emp_ID     Salary
-----------------------------------------------
1     Anita                3002       52000.50
2     Bimal                3003       47000.00
3     Charan               3005       43000.75
4     Deepa                3004       47000.00
5     Esha                 3006       60000.00
6     Farhan               3007       39000.00
7     Gita                 3008       52000.50
8     Hema                 3009       47000.00
9     Ishan                3010       48000.25
10    Ravi                 3001       45000.00

Merge sort execution time: 55 micro-secs
manavaryasingh@MacBook-Pro-5 kritika-dsa %
```